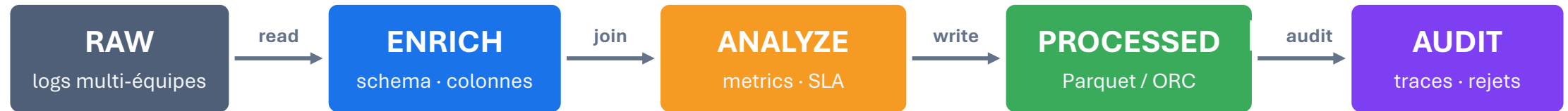


Spark

Cas d'usage DORA

De logs bruts à des indicateurs auditables



DataFrame · Schema · Shuffle · Join · Broadcast · Partitioning · Quality · Performance

Cours 1h30 · TP 2h · Projet fil rouge DORA

Where Are We?

Trois responsabilités distinctes dans notre plateforme

HDFS

STORAGE

- Fichiers distribués
- Blocs et réplication
- Data Lake

YARN

RESOURCES

- CPU / mémoire
- Containers
- Scheduling

SPARK

PROCESSING

- Transformations
- Jointures
- Agrégations

Spark orchestre le traitement ; HDFS stocke les données ; YARN fournit les ressources.

Session 4 → Session 5

De comprendre Spark à concevoir un pipeline

SESSION 4

Comment Spark exécute ?

SparkSession

DataFrame

Transformation vs Action

Lazy evaluation

Spark sur YARN

progression

SESSION 5

Comment construire un pipeline robuste ?

Schéma explicite

Enrichissement

Jointures

Qualité

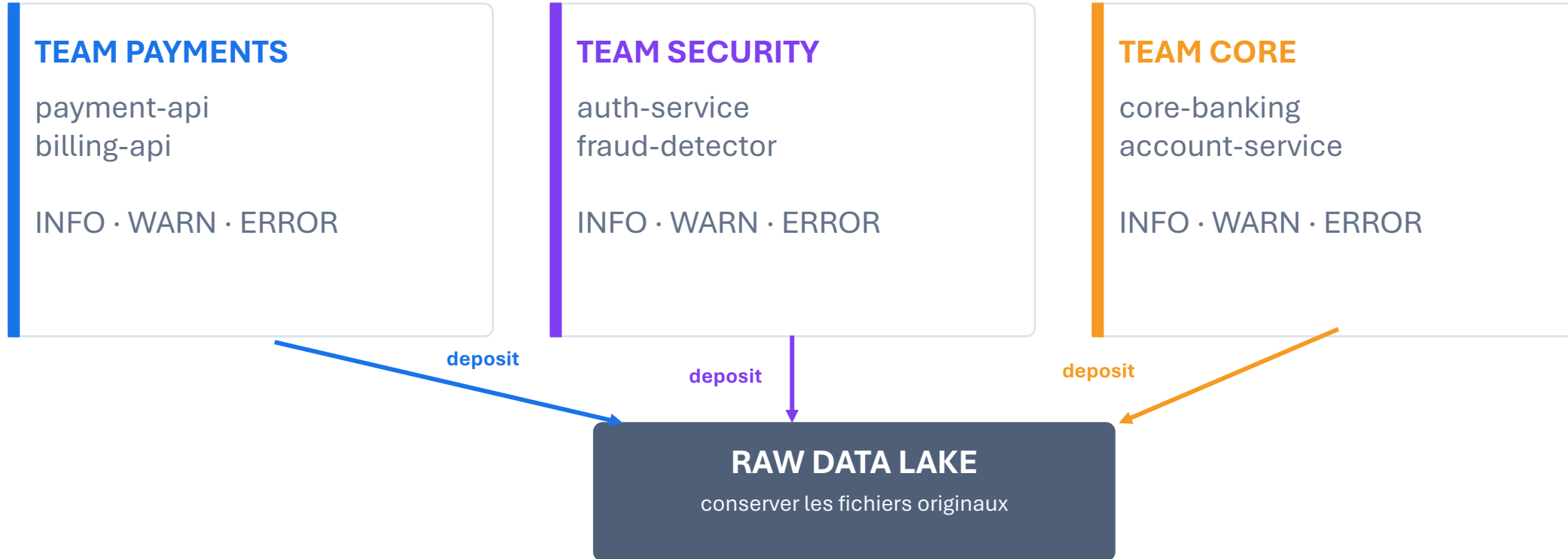
Partitionnement

Auditabilité

Aujourd'hui : penser comme un Data Engineer, pas seulement comme un utilisateur d'API.

The DORA Data Problem

Plusieurs équipes produisent des logs techniques



Le pipeline doit préserver la provenance des données et produire des indicateurs fiables.

Our Target Pipeline

Le fil rouge de toute la séance



Data contract

Lineage

Business context

Audit evidence

Chaque flèche représente un flux de données réel.

Structured Data in Spark

Le DataFrame donne une structure exploitable au moteur Spark

```
event_ts,app_id,level,status_code,response_time_ms  
2026-01-15T08:02:15Z,payment-api,ERROR,504,2100
```

read
→

DataFrame

```
event_ts : string  
app_id : string  
level : string  
status_code : int  
response_time_ms : int
```

Named columns

Typed schema

Distributed rows

Spark peut raisonner sur les colonnes, les types et le plan de calcul.

DataFrame vs Dataset

Même moteur, APIs différentes

Dimension	DataFrame	Dataset typé
Structure	Colonnes + schéma	Objets typés
Langages	Python · Scala · Java · R	Scala · Java
Typage compile-time	Non	Oui côté JVM
Dans ce module	✓ PySpark	Concept à connaître

Notre TP : PySpark DataFrame API

En PySpark, le modèle principal est le DataFrame.

Why Does the Schema Matter?

Un type incorrect peut transformer une donnée valide en résultat faux

WITHOUT CONTROL

`response_time_ms = "2100"`

`status_code = "504"`

`event_ts = "..."`

Types inférés ou incohérents

EXPLICIT SCHEMA

`response_time_ms : int`

`status_code : int`

`event_ts : string`

Types attendus et documentés

BUSINESS EFFECT

`avg(response_time_ms)`

SLA breach

Data quality

Résultats reproductibles

Le schéma devient un contrat entre producteurs et consommateurs.

Explicit Schema vs inferSchema

Le schéma explicite réduit l'ambiguïté

inferSchema

- ✓ Rapide pour explorer
- ✗ Dépend du contenu observé
- ✗ Types parfois inattendus
- ✗ Moins contrôlé pour un pipeline stable

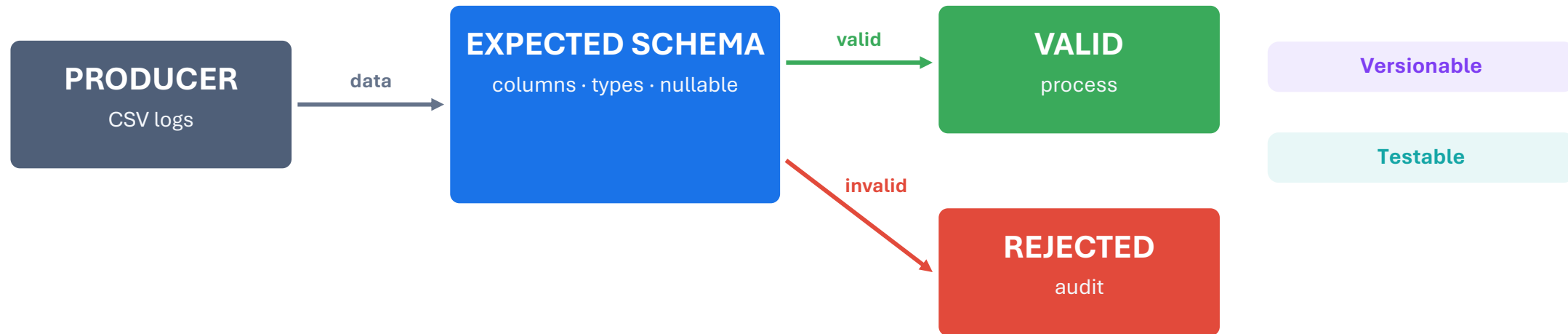
.schema(log_schema)

- ✓ Contrat explicite
- ✓ Types prévisibles
- ✓ Reproductibilité
- ✓ Qualité plus simple à contrôler

Le TP impose StructType pour les logs et les référentiels.

Schema = Data Contract

Le pipeline sait ce qu'il accepte et ce qu'il rejette

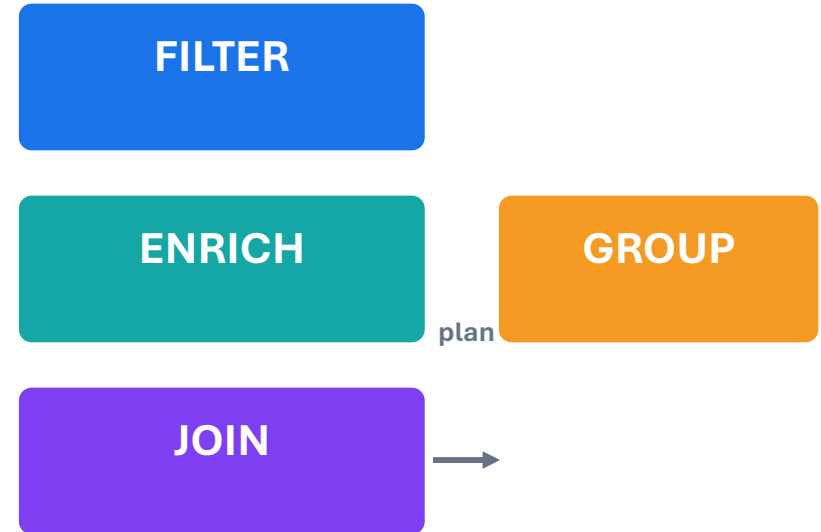


Un schéma explicite rend les attentes vérifiables.

Transformations Build a Plan

Une transformation décrit ce que l'on veut obtenir

```
logs
  .filter(F.col("env") == "prod")
  .withColumn("is_error", ...)
  .join(apps, "app_id", "left")
  .groupBy("event_date", "app_id")
```



Pas encore de résultat final demandé

Spark construit une lignée de transformations avant l'action.

Actions Trigger Execution

Une action oblige Spark à produire un résultat

```
df.show()  
  
df.count()  
  
df.write.parquet(path)
```

trigger

EXECUTION

Logical plan

↓

Optimized plan

↓

Physical plan

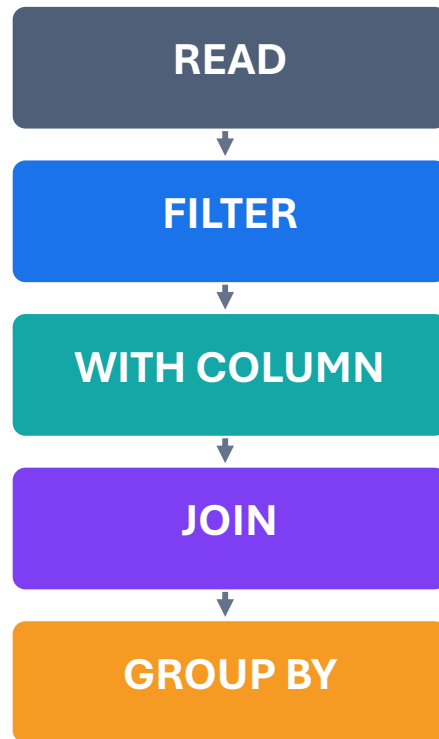
↓

Jobs · Stages · Tasks

`show()`, `count()` ou `write()` peuvent déclencher un ou plusieurs jobs.

One Pipeline, One Logical Plan

Le code est une description ; Spark décide comment l'exécuter



SPARK SQL ENGINE

- Analyse du plan
- Optimisation
- Choix des opérateurs physiques
- Planification des échanges réseau
- Génération des stages

La chaîne de transformations devient un plan optimisable.

Narrow vs Wide Transformations

Toutes les transformations n'ont pas le même coût réseau

NARROW

filter
select
withColumn

Données traitées principalement dans les partitions existantes.

→ peu ou pas d'échange réseau global

WIDE

groupBy
repartition
many joins

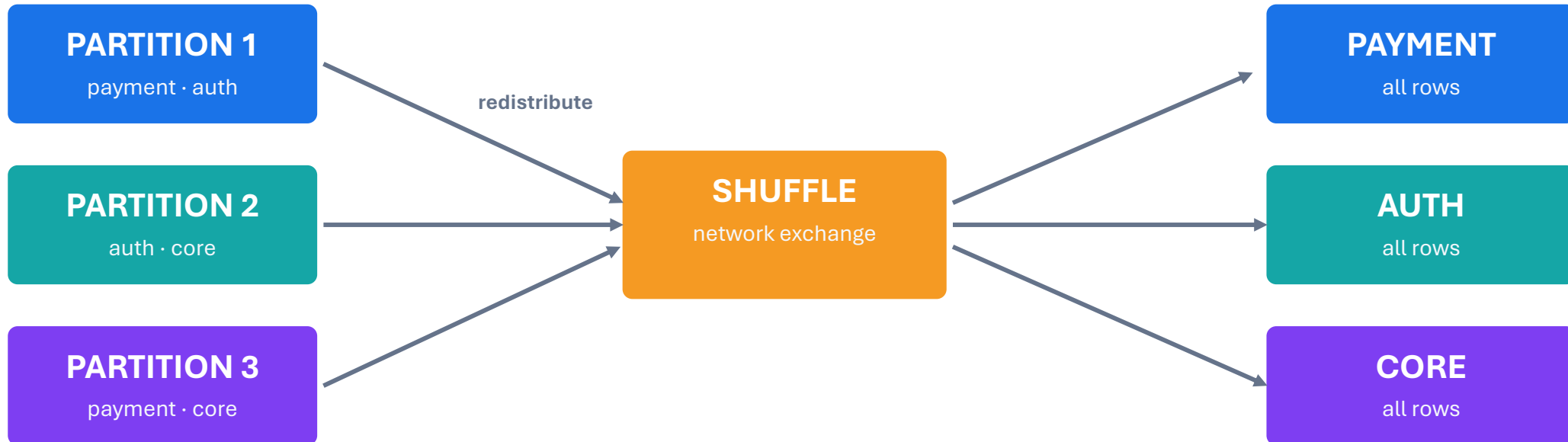
Les lignes peuvent changer de partition.

→ échange réseau possible

Une opération « wide » peut nécessiter une redistribution entre partitions.

The Enemy Has a Name: Shuffle

Le shuffle redistribue des données entre executors



Un shuffle n'est pas « mauvais » - il doit être compris et maîtrisé.

Multiple Producers, One Data Lake

La zone raw doit séparer les producteurs sans perdre la flexibilité

```
/raw/  
team_payments/application_logs/year=2026/month=01/day=15/  
team_security/application_logs/year=2026/month=01/day=15/  
team_core/application_logs/year=2026/month=01/day=15/  
referentials/
```

Provenance

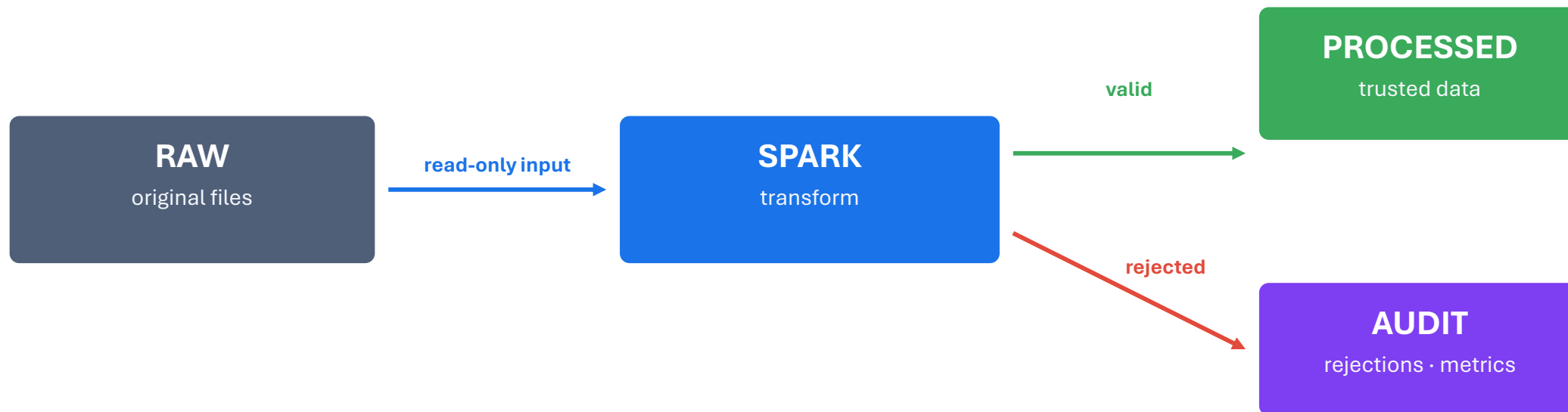
Isolation équipes

Organisation temporelle

Le TP utilise une arborescence par équipe puis par date.

Raw Means Raw

La zone raw conserve la preuve originale



Ne pas écraser les fichiers raw

Transformer les données ne doit pas détruire la source brute.

Trace the Data Origin

Le chemin source devient une métadonnée du pipeline



Le TP extrait `source_file` puis `source_team` à partir du chemin HDFS.

Derive Business Columns

Transformer un log technique en attributs directement analysables

Colonne source	Transformation	Colonne dérivée
event_ts	to_timestamp()	event_timestamp
event_timestamp	to_date()	event_date
event_timestamp	hour()	event_hour
level == ERROR	boolean	is_error
response_time_ms	when()	latency_bucket

Le TP crée date, heure, flags d’erreur et classe de latence.

Why Derive Columns?

Une valeur technique devient une information exploitable

```
2026-01-15T08:02:15Z,payment-api,ERROR,504,2100
```

derive

DERIVED VIEW

```
event_date = 2026-01-15  
event_hour = 08  
is_error = true  
latency_bucket = slow
```

Filter

Aggregate

Partition

Audit

On prépare les données pour le calcul des indicateurs.

Logs Are Not Enough

Les référentiels ajoutent le contexte métier

TECHNICAL LOG

app_id = payment-api
level = ERROR
status = 504
response = 2100 ms

APP REFERENCE

app_name
owner_team
criticality
business_domain

SLA REFERENCE

max_response_time_ms
sla_level

Un app_id seul ne dit pas qui possède l'application ni son niveau de criticité.

From Technical Log to Business Context

L'enrichissement transforme un événement en information métier

BEFORE

payment-api
ERROR
504
2100 ms

join refs

AFTER

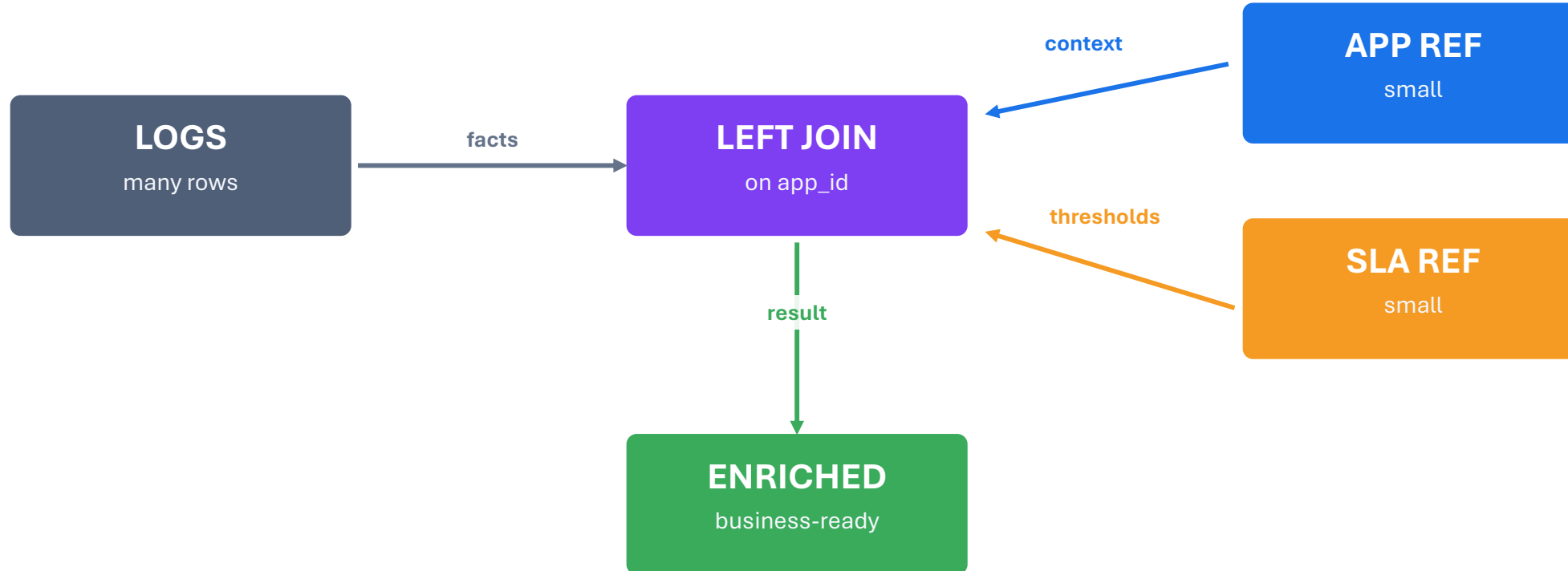
Payment API
Owner = team_payments
Criticality = high
SLA = gold · 800 ms
Response = 2100 ms

SLA BREACHED = TRUE

On peut désormais mesurer criticité, responsabilité et SLA.

Joining Logs with Reference Data

Une jointure relie les faits techniques au contexte de référence



La clé de jointure du TP est app_id.

Why a LEFT JOIN?

Ne pas perdre un log simplement parce qu'un référentiel est incomplet

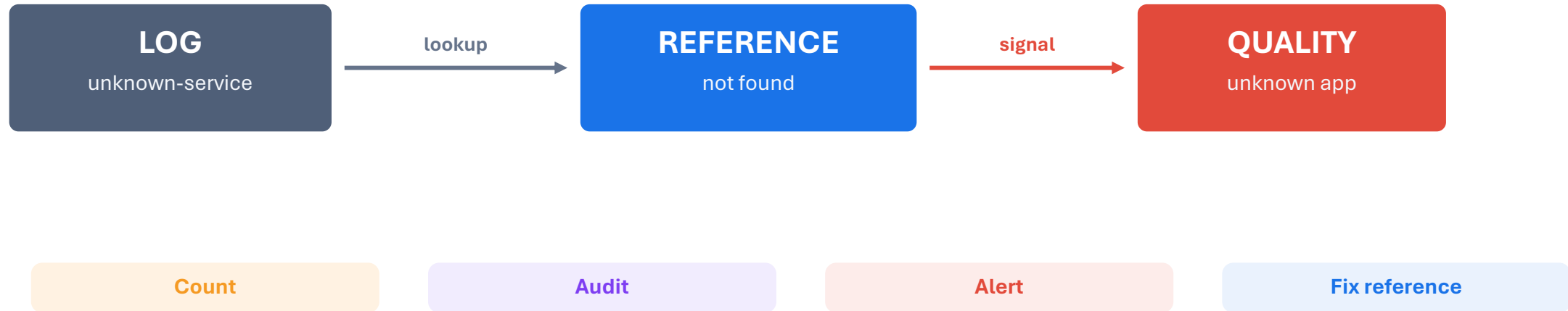
app_id dans logs	Référence trouvée ?	LEFT JOIN result
payment-api	Oui	Payment API
auth-service	Oui	Authentication Service
unknown-service	Non	app_name = NULL

NULL ≠ à ignorer : c'est une information

Une absence de référence devient un signal de qualité.

Unknown Application = Data Quality Signal

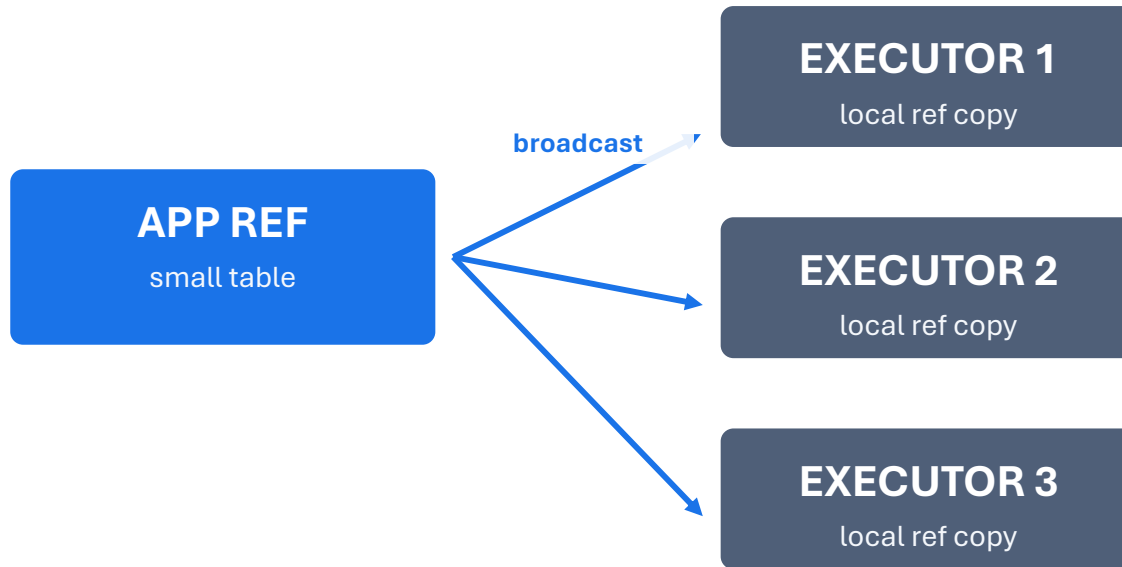
Une absence de référentiel doit être visible et traitable



Le pipeline peut compter, auditer et alerter sur les applications inconnues.

Broadcast Join

Diffuser un petit référentiel plutôt que redistribuer le gros dataset



BIG LOG DATASET

Les partitions de logs restent distribuées.

Chaque executor peut joindre localement avec la petite table diffusée.

Objectif : limiter l'échange réseau.

Le TP force `F.broadcast(apps)` et `F.broadcast(sla)`.

When NOT to Broadcast

Le broadcast consomme de la mémoire sur chaque executor

GOOD CANDIDATE

Référentiel applications
Quelques lignes / quelques Mo

- ✓ Copie sur chaque executor
- ✓ Jointure locale rapide

BAD CANDIDATE

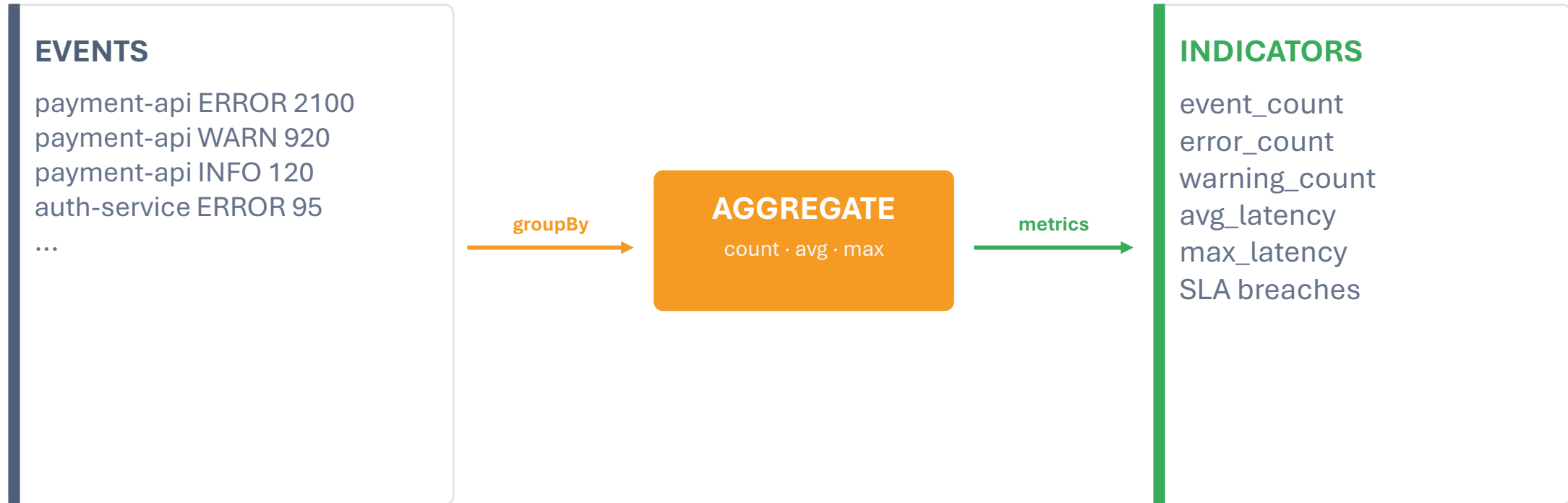
Historique de logs
Plusieurs Go / To

- ✗ Pression mémoire
- ✗ Risque OOM
- ✗ Coût de diffusion

Une petite table oui ; un gros dataset non.

Raw Events → Metrics

Les événements individuels deviennent des indicateurs



L'agrégation répond à une question métier précise.

Daily Metrics by Application

Le TP produit une vue quotidienne par application

Dimension / Measure	Exemple
event_date	2026-01-15
app_id	payment-api
event_count	3
error_count	1
avg_response_time_ms	1046.7
sla_breach_count	2
error_rate	0.333

Les métriques deviennent directement exploitables pour analyse et audit.

Count vs Rate

Un nombre brut peut masquer la gravité relative

APPLICATION A

1 000 événements
100 erreurs

ERROR RATE = 10 %

APPLICATION B

100 événements
50 erreurs

ERROR RATE = 50 %

Contexte > nombre brut

Le taux d'erreur contextualise le nombre d'erreurs.

Application Metrics vs Team Metrics

Changer le niveau d'agrégation change la question

APPLICATION VIEW

payment-api
auth-service
core-banking
...

Question : quelle application est la plus dégradée ?

TEAM VIEW

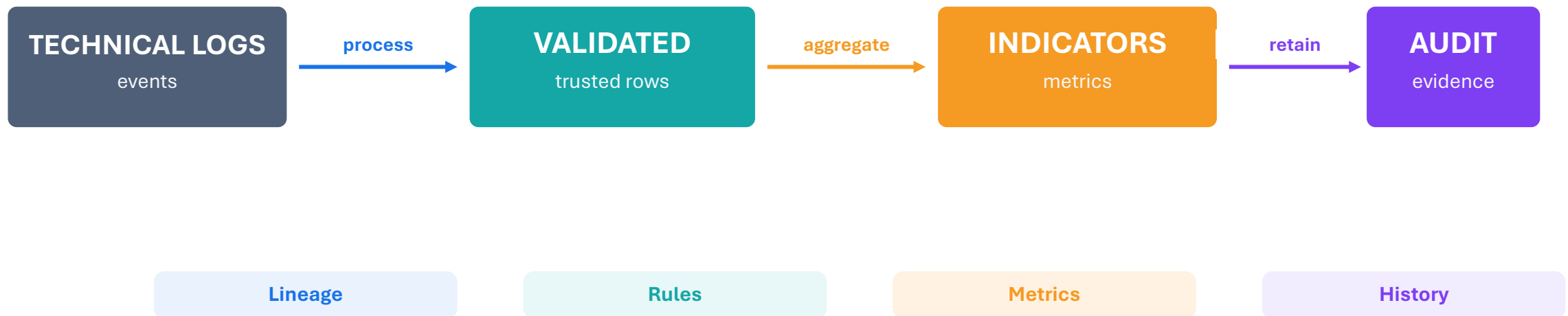
team_payments
team_security
team_core

Question : quelle équipe produit le plus d'erreurs / de volume ?

Le TP produit les deux vues.

From Operational Metrics to Audit Evidence

Un indicateur d'audit doit être traçable et reproductible



Le projet demande la production d'indicateurs d'audit à partir des logs.

CSV Is Good for Exchange - Not Always for Analytics

Le format de stockage influence fortement la lecture et le coût

CSV

- ✓ Simple à produire
- ✓ Lisible humainement
- ✓ Très interopérable
- ✗ Texte
- ✗ Types non embarqués de la même manière
- ✗ Lecture analytique moins efficace

PARQUET / ORC

- ✓ Colonnaire
 - ✓ Compression
 - ✓ Schéma
 - ✓ Lecture sélective de colonnes
 - ✓ Bon fit analytique
- zone processed

Le TP utilise CSV en raw puis Parquet / ORC en processed.

Why Columnar Storage?

Une requête analytique ne lit souvent qu'une partie des colonnes

event_ts	app_id	env	level	status	response_time_ms	message
...	payment-api	prod	ERROR	504	2100	timeout
...	auth-service	prod	INFO	200	80	token issued



Query needs only 2 columns

Le format colonne réduit les données à lire lorsque seules quelques colonnes sont nécessaires.

Partitioning the Output

Organiser physiquement les fichiers selon les dimensions de lecture

```
events_parquet/  
└─ event_date=2026-01-15/  
   └─ app_id=payment-api/  
      └─ part-00000-....parquet  
      └─ part-00001-....parquet  
   └─ app_id=auth-service/  
      └─ part-00000-....parquet
```

DESIGN QUESTIONS

Quelles colonnes filtrons-nous souvent ?

Combien de valeurs distinctes ?

Quel volume par partition ?

Risque de petits fichiers ?

Le TP écrit les logs enrichis par event_date puis app_id.

Partition Pruning

Spark peut éviter de scanner les répertoires inutiles

```
df.filter(  
    (F.col("event_date") == "2026-01-15")  
&  
    (F.col("app_id") == "payment-api")  
)
```

FILESYSTEM VIEW

event_date=2026-01-14 ✗
event_date=2026-01-15
 app_id=auth-service ✗
 app_id=payment-api ✓
event_date=2026-01-16 ✗

Le TP vérifie ce comportement avec un filtre + `explain(True)`.

Partitioning Is a Design Decision

Trop peu ou trop de partitions créent des problèmes différents

TOO FEW

Quelques partitions énormes

- faible parallélisme
- gros scans
- écriture concentrée
- potentiel skew

TOO MANY

Des milliers de petits fichiers

- overhead HDFS
- listing coûteux
- nombreuses tasks
- métadonnées

Chercher un compromis adapté au volume

Le petit fichier HDFS réapparaît ici.

Three Different “Partitions”

Même mot, trois concepts à ne pas confondre

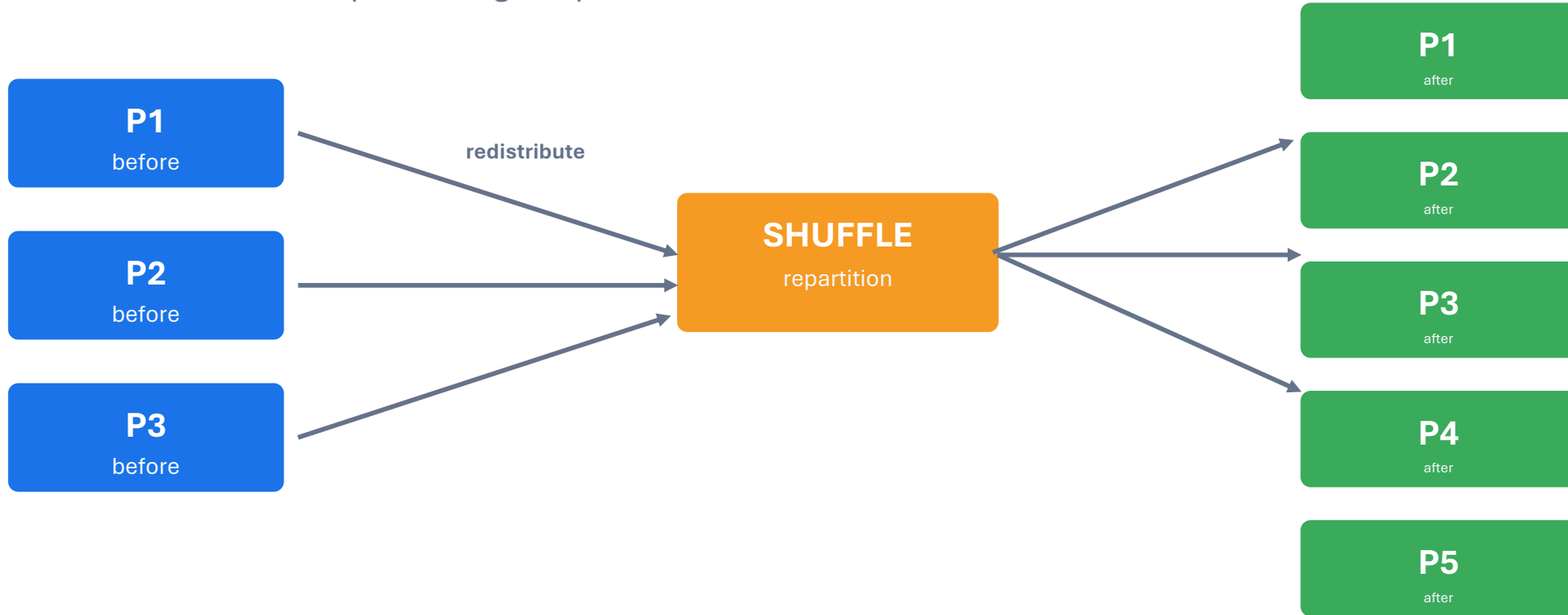
Concept	Rôle	Exemple
HDFS block	Stockage physique distribué	128 MB block
Spark partition	Unité de parallélisme / traitement	1 task traite 1 partition à la fois
partitionBy()	Organisation des fichiers de sortie	event_date=.../app_id=.../

Même terme ≠ même abstraction

Cette distinction évite beaucoup d’erreurs en Spark/Hadoop.

repartition()

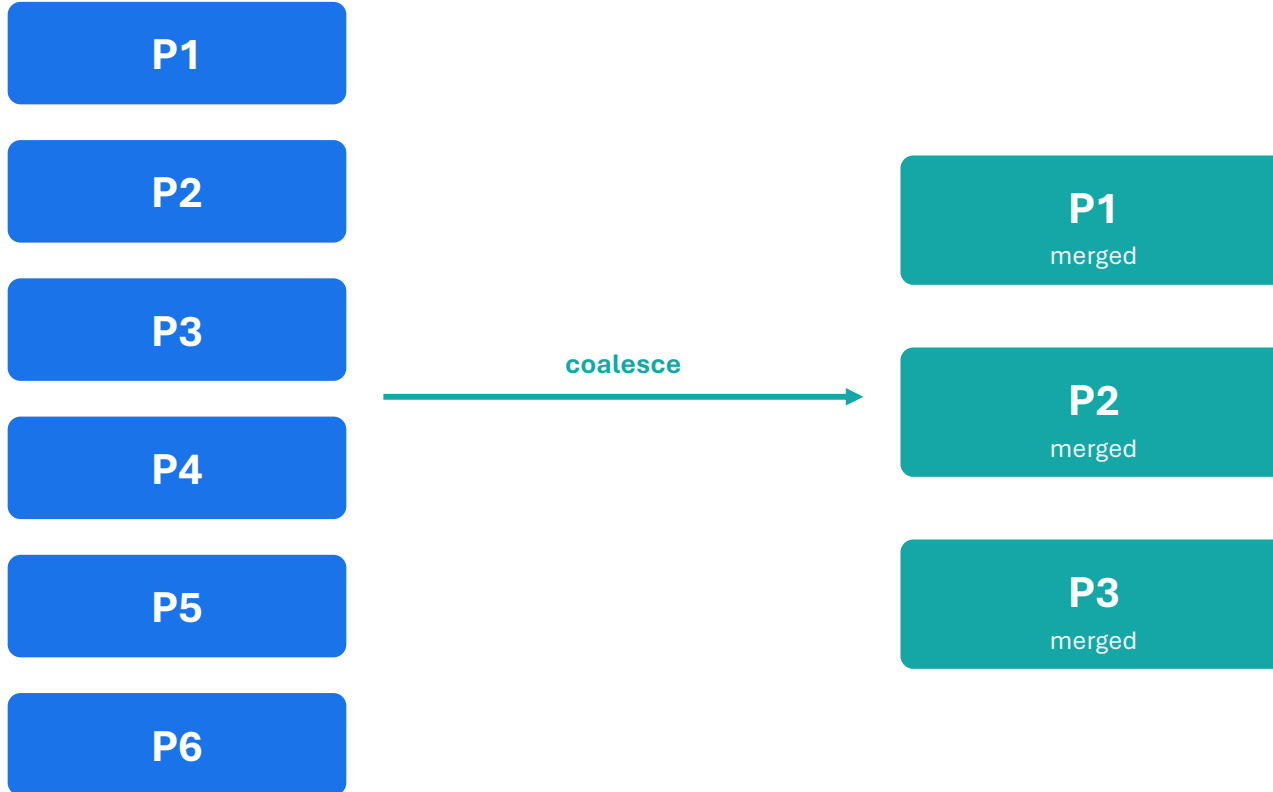
Redistribuer les données pour changer le partitionnement



`repartition()` provoque typiquement un shuffle.

coalesce()

Réduire le nombre de partitions avec moins de redistribution



KEY IDEA

Réduire : oui

Shuffle complet : souvent non

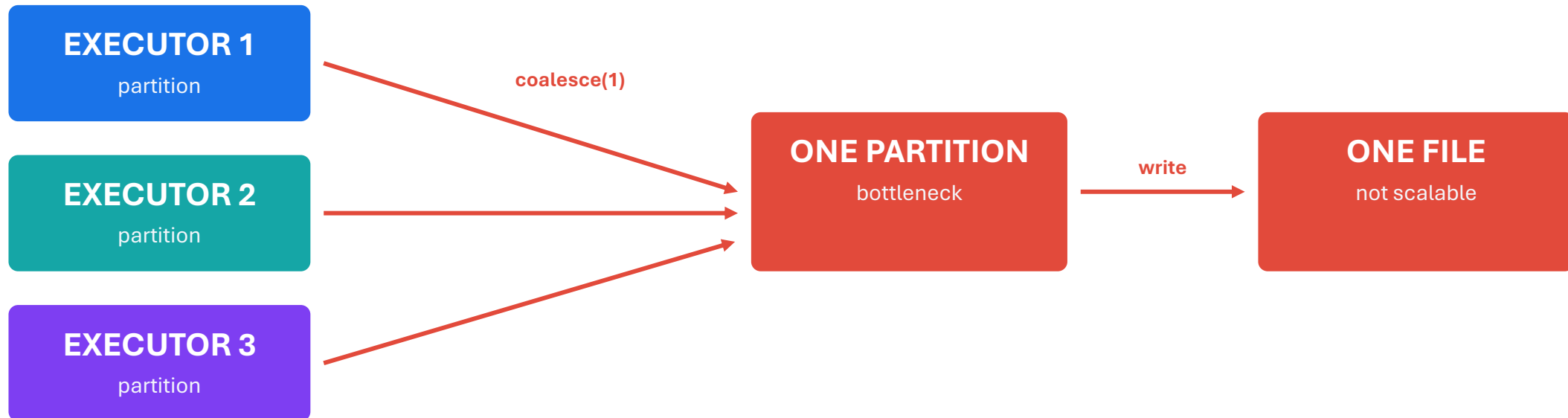
Augmenter les partitions : préférer `repartition()`

Cas extrême : `coalesce(1)` = parallélisme réduit

`coalesce()` est souvent utile pour réduire les partitions sans shuffle complet.

Why coalesce(1) Is Not a Production Strategy

Un seul fichier peut devenir un goulot d'étranglement



Le TP demande explicitement d'analyser ce risque.

Performance Is About Trade-offs

Il n'existe pas de nombre magique de partitions

PARALLELISM

Assez de partitions pour occuper les executors

SHUFFLE

Éviter les redistributions inutiles

FILE SIZE

Éviter trop petits ou trop gros fichiers

MEMORY

Cache, broadcast, agrégations

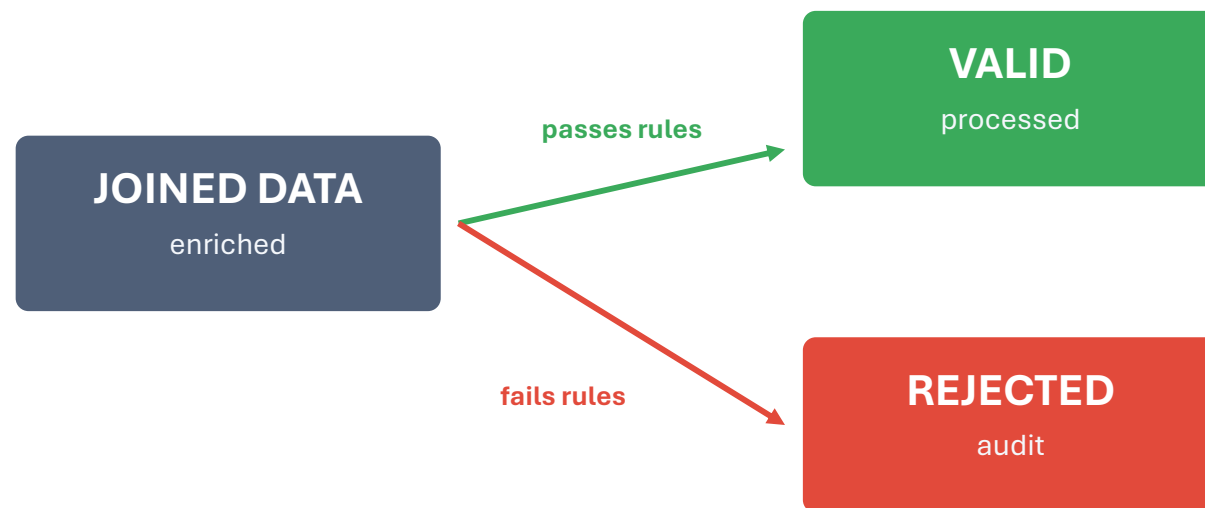
NETWORK

Shuffle et lecture distante

Tuning = observer, mesurer, ajuster.

Valid vs Rejected

La qualité doit être explicite dans le pipeline



QUALITY RULES

event_timestamp != NULL
app_id != NULL
status_code != NULL
response_time_ms != NULL

+ raison du rejet
+ source_team
+ process_date

Le TP sépare les lignes valides et rejetées avant écriture.

Never Silently Drop Bad Data

Supprimer une ligne invalide détruit une information utile

BAD PATTERN

```
response_time_ms = NULL
```

- drop row
- no trace
- no explanation
- impossible d'auditer

BETTER PATTERN

```
response_time_ms = NULL
```

- REJECT
- reason = missing_response_time
- source = team_payments
- processing_date
- audit output

Une ligne rejetée doit rester explicable.

Auditability Requires Lineage

Pouvoir expliquer six mois plus tard comment un indicateur a été produit



Question d'audit : “Comment ce KPI a-t-il été calculé ?”

La provenance doit traverser tout le pipeline.

Raw / Processed / Audit

Trois zones, trois responsabilités

RAW

Original evidence

- multi-format
- provenance
- immutable mindset
- reprocessing source

PROCESSED

Trusted analytical data

- enriched
- typed
- Parquet / ORC
- partitioned

AUDIT

Evidence & control

- metrics
- rejected rows
- execution metadata
- lineage

Une architecture claire évite de mélanger preuve brute, données de travail et résultats d'audit.

Your Code Is Not the Execution Plan

Le code DataFrame est déclaratif ; Spark choisit les opérateurs physiques

```
df
  .join(F.broadcast(apps), "app_id", "left")
  .groupBy("event_date", "app_id")
  .agg(...)
```

≠

PHYSICAL PLAN

FileScan
BroadcastExchange
BroadcastHashJoin
Exchange
HashAggregate
WriteFiles

Pour comprendre la performance, il faut regarder le plan.

explain()

Le premier outil pour comprendre ce que Spark prévoit de faire

```
df.filter(  
    (F.col("event_date") == "2026-01-15") &  
    (F.col("app_id") == "payment-api")  
) .explain(True)
```

WHAT TO LOOK FOR

1. FileScan
2. PartitionFilters
3. PushedFilters
4. Broadcast / SortMerge Join
5. Exchange = shuffle
6. HashAggregate
7. Output write

Le TP utilise `explain(True)` sur la lecture partitionnée.

Spark UI / History Server

Observer l'application au lieu de deviner

APPLICATION

Jobs

Stages

Tasks

SQL / DAG

EXECUTORS

CPU time

Memory

Input / Output

Shuffle read/write

CLUSTER

YARN app status

Containers

Logs

Resources

Le TP demande d'identifier jobs, stages, tasks, shuffles et I/O.

Diagnosing a Slow Spark Job

Performance tuning starts with evidence

CPU

Tasks longues
Utilisation CPU élevée

→ calcul coûteux

MEMORY

GC
Spill
OOM

→ partitions / cache /
broadcast

SHUFFLE

Shuffle read/write élevé

→ groupBy / join /
repartition

DATA SKEW

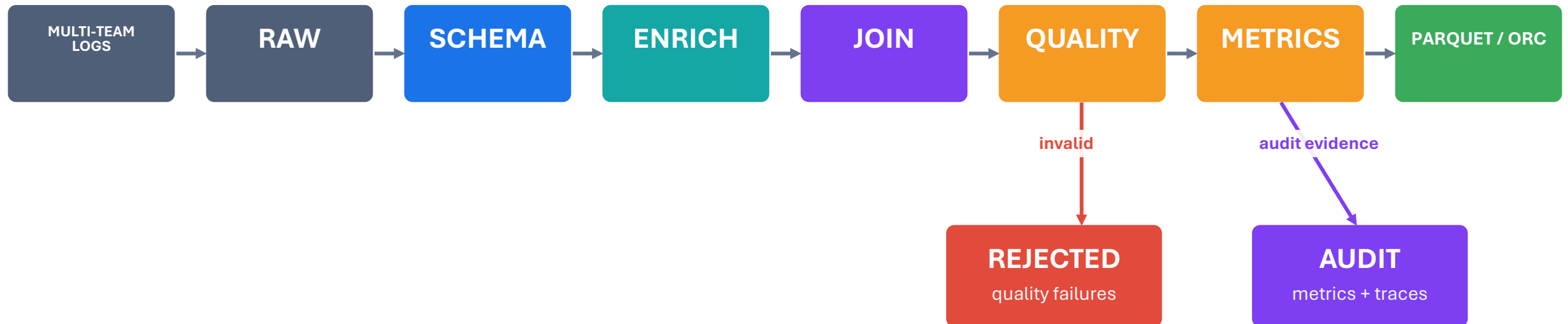
Quelques tasks très
lentes

→ clés déséquilibrées

Commencer par identifier où le temps est dépensé.

The Complete DORA Spark Pipeline

Tous les concepts de la séance dans un seul flux



Chaque flèche représente un vrai flux de données.

What Makes This a Production-Like Pipeline?

La robustesse vient de plusieurs décisions cohérentes

Schema contract

Lineage

Reference data

Quality controls

Audit trail

Efficient formats

Partition strategy

Monitoring

Reproducibility

Un pipeline de production est plus qu'une suite de transformations.

TP05 - Spark Advanced Logs

6 missions pour construire le pipeline complet

1 · INGEST

Logs multi-équipes
Zones HDFS raw / processed / audit

2 · STRUCTURE

Schéma explicite
source_file / source_team

3 · ENRICH

Colonnes dérivées
Référentiels + broadcast joins

4 · ANALYZE

Erreurs · latence · SLA
Agrégations app / équipe

5 · PERSIST

Parquet / ORC
Partitioning · repartition · coalesce

6 · OPERATE

Quality · rejected rows
YARN · Spark UI · History Server

```
example-spark-project/  
├─ jobs/log_pipeline_job.py  
└─ src/log_pipeline/{io,schemas,transforms,quality,metrics}.py
```

Le TP va de l'ingestion multi-équipes au monitoring du job Spark.